

- 🔥 Day 10 — MEMORY + CONTEXT (clean integration with YOUR Day 9 system)
- 👉 This version plugs directly into the planner + Claude Excel agent you already have.
-

✅✅ WHAT YOU ADD TODAY

Planner ✅ (Day 9) + Memory ✅ (Day 10) = Conversational Agent 🔥

✅✅ STEP 1 — CREATE MEMORY FILE

📌 Create: app/memory.py

✅ Code:

```
class Memory:
    def init(self, max_turns=5):
        self.history = []
        self.max_turns = max_turns

    def add(self, query, response):
        self.history.append((query, response))
        # keep only recent history
        if len(self.history) > self.max_turns:
            self.history.pop(0)

    def get(self):
        context = ""
        for q, r in self.history:
            context += f"User: {q}\nAssistant: {r}\n"
        return context
```

✅✅ STEP 2 — UPDATE agent.py

📌 File: app/agent.py

✅ Modify imports:

```
from app.memory import Memory
from app.planner import Planner
from models.llm import generate
```

✅ Update class:

```
class Agent:
    def init(self, rag_system, excel_tool=None):
        self.rag = rag_system
        self.excel = excel_tool
        self.planner = Planner()
        self.memory = Memory() # ✅ NEW
```

✅ Replace FULL run() with this:

```
def run(self, query):
    # ✅ Step 1: get past conversation
    past_context = self.memory.get()
    # ✅ enrich query with memory
    enriched_query = f"""
Conversation so far:
{past_context}
Current question:
{query}"""
    # ✅ Step 2: planning
    plan = self.planner.create_plan(enriched_query)
    if not plan:
        plan = [("EXCEL", query)]
```

```

print("FINAL PLAN:", plan)

results = []

# ✅ Step 3: execute plan
for i, (tool, task) in enumerate(plan, 1):

    print(f"Executing Step {i}: {tool} → {task}")

    if tool == "EXCEL" and self.excel:

        result = self.excel.smart_query(task, generate)

    elif tool == "RAG":

        docs = self.rag.search(task)

        result = "\n".join(docs[:3])

    else:

        result = "Unknown tool"

    print(f"STEP {i} RESULT:", result)

    results.append(f"Step {i} Result:\n{result}")

# ✅ Step 4: final answer (with memory)
final_prompt = f"""
Conversation:
{past_context}

Results: {results}

User question: {query}

Give final answer: """

final_answer = generate(final_prompt)

# ✅ Step 5: save memory
self.memory.add(query, final_answer)

return final_answer

```

✅✅ STEP 3 — RUN

```
python -m app.main
```

✅✅ TEST LIKE THIS

✅ Conversation test

>> what is average price 👉 Answer: ✅

>> what about year 2015

👉 It understands previous context ✅🔥

>> and lowest km?

👉 Continues conversation ✅

✅✅ WHAT YOU JUST BUILT

Before:

❌ Stateless agent

(no memory)

Now:

✅ Conversational multi-step agent

✅ remembers past queries

✅ handles follow-ups

✅ behaves like ChatGPT ✅🔥

✅✅ YOUR SYSTEM NOW

✓ PDF RAG ✅

✓ Excel Agent ✅

✓ Tool Routing ✓

✓ Multi-step Planner ✓

✓ Memory ✓ 🔥

🚀 NEXT (your earlier question)

👉 Day 11 = PDF TABLE EXTRACTION

PDF tables → DataFrame → same Excel agent ✓

💬 Say Day 11 when ready — that's your missing piece 🔥